Presentation script (10 minutes)

ECG classification when leads go missing · Wang Youyang (G2606821F) · MD6329 Deep Learning for Healthcare

Word-for-word spoken script for `01_ecg_lead_masking.py` (43 steps). Each ▶ **Step N** line means: press → (or the › button) now; the text after it is what you say while that step is on screen. Timings assume about 130 words per minute (about 1,350 words in total, roughly 10 minutes).

Before you start: run `lectrace serve 01_ecg_lead_masking.py`, open `http://localhost:7000`, open the variables panel (split-panel icon, top right), and go to step 1.

Opening · 0:00–2:35 · steps 1–6

> ▶ **Step 1** — the file opens at `def main()`. Press on straight away.

> ▶ **Step 2** — title: *ECG classification when leads go missing*

Hi everyone. My project is about ECG classification when some of the leads are missing.

> ▶ **Step 3** — the paragraph about the reading service

Picture a remote ECG reading service. On the same day it might get a full twelve-lead ECG from a hospital, a two-lead recording from an ambulatory monitor, and a six-lead recording from a small handheld device that a patient uses at home. Devices like that are becoming common.

Most ECG models are trained on complete twelve-lead ECGs. With fewer leads their accuracy can drop, and nothing in the output tells the doctor that the result is less reliable.

Some of that drop is fair, because fewer leads means less information. But some of it isn't. The six limb leads are not independent: if you know any two of them, you can calculate the other four exactly. So if a model fails with only leads one and two, the problem isn't missing information, and that is what this talk is about.

> ▶ **Step 4** — the question and the design table

My question has two parts. First, if we train one classifier on a mix of complete ECGs and ECGs with random leads removed, does it handle missing leads better than the same model trained only on complete ECGs? And second, why does the complete-input model fail in the first place?

The design is a matched comparison. Model A always sees complete inputs. Model B sees a complete input half of the time, and the other half a random subset of leads, with the rest set to zero. Everything else is identical: the network, the data, the training time, the random seeds, and how the final checkpoint is chosen. So any difference comes from the masking.

The data is PTB-XL, a public set of about twenty-two thousand ECGs, with five diagnostic classes: normal, myocardial infarction, ST/T changes, conduction problems and hypertrophy.

▶ **Step 5** — the link to Reyna et al.

The test inputs come from the 2021 PhysioNet challenge: all twelve leads, the six limb leads, two small sets with chest lead V2, and finally just leads one and two, which we'll call S2.

▶ **Step 6** — presenter note (don't read it aloud)

Now I'll walk through four moments in the computation that explain the results.

State 1 · 2:35–4:20 · steps 7–16 · the input

▶ **Step 7** — heading: *State 1 — a zero-filled ECG breaks a physical law*

Let's start with the input, before the network does anything.

▶ **Step 8** — panel: `ecg_id = 514`

This is one ECG from the test set, record 514. I picked it with a fixed rule before running any model: it's the first test record whose only diagnosis is an inferior heart attack.

▶ **Step 9** — panel: `einthoven_residual = 0.002`

Here I check Einthoven's law: lead three should equal lead two minus lead one. On the real recording the error is 0.2 percent, so the law holds.

▶ **Step 10** — panel: `available = ["I", "II"]`

Now we simulate the two-lead setting. Only leads one and two are kept.

▶ **Step 11** — the zero-filling line runs

The missing leads are filled with zeros. That's the usual way to give a network a missing input.

▶ **Step 12** — panel: `zero_fill_residual = 1`

And now the error is one hundred percent. To the network, lead three is a flat line, which goes against what it saw in every single training ECG.

▶ **Step 13** — the derivation line runs

But we don't have to use zeros. We can calculate the missing limb leads from leads one and two.

▶ **Step 14** — panel: `derived_residual = 0.002`

And the error is back to 0.2 percent.

▶ **Step 15** — the lead III chart

Here's the picture. The grey line is the real lead three, and the blue dashed line is the one we calculated. They sit right on top of each other. The red line is what the network gets with zeros.

▶ **Step 16** — the summary text

So with only leads one and two, almost all the information is still there. What breaks it is the way we fill in the missing leads.

State 2 · 4:20–6:35 · steps 17–25 · inside the forward pass

▶ **Step 17** — heading: *State 2 — inside the forward pass*

Now let's look inside the network. This is the most important part of the talk.

The network uses batch normalization after every convolution. During training, each batch-norm layer learns the usual mean and spread of its inputs. At test time it uses those stored numbers to normalize. That works, as long as the test inputs look like the training inputs.

▶ **Step 18** — panel: `shift_A_complete`

For every batch-norm layer, I measure how far the average of the test activations drifts from the stored average, in standard deviations. For model A with all twelve leads, the drift is tiny, at most 0.04. Everything matches.

▶ **Step 19** — panel: `shift_A_S2`

Now the same model A with only leads one and two. The drift jumps to between 0.23 and 0.45, in every layer. So every layer now normalizes with the wrong numbers.

▶ **Step 20** — panel: `shift_B_S2`

Model B, with the same two leads, drifts less, between 0.09 and 0.25. B was trained on batches that already mixed complete and masked ECGs, so its stored numbers expect missing leads.

▶ **Step 21** — the batch-norm chart

Here it is across all eleven layers. Orange is A with all leads, close to zero. Blue is A with two leads, high everywhere. And red is B, in between.

▶ **Step 22** — panel: `auroc_A`

So what does this do to the output? I compute the AUROC for each class on the whole test set. AUROC is the chance that the model ranks a sick patient above a healthy one, so 0.5 is a coin flip. With two leads, model A gets 0.35 for normal, 0.40 for heart attack, and 0.38 for hypertrophy.

▶ **Step 23** — panel: `auroc_B`

Model B gets between 0.81 and 0.91.

▶ **Step 24** — panel: `below_chance = ["NORM", "MI", "HYP"]`

Three of A's five classes are below 0.5. So A isn't just guessing. It's ranking them backwards.

▶ **Step 25** — the summary text

So a big part of A's failure is simply batch-norm statistics that no longer fit the input.

State 3 · 6:35–7:55 · steps 26–32 · the repair

▶ **Step 26** — heading: *State 3 — repair A at test time*

If that's the problem, can we fix model A without retraining it?

▶ **Step 27** — the re-estimation line runs (a few seconds of computation)

Here's the repair, in two steps. First, we calculate the missing limb leads, like in state one. Second, we re-estimate every batch-norm mean and variance by passing 4,096 training ECGs through the model, with the same leads removed. No weight changes at all.

▶ **Step 28** — panel: `macro_zero = 0.483`

Before the repair, model A's average AUROC with two leads is 0.483.

▶ **Step 29** — panel: `macro_repaired = 0.842`

After the repair, it's 0.842. Almost all of the drop comes back, without any training.

▶ **Step 30** — panel: `share_closed`

Over all four reduced-lead settings and all three seeds: calculating the leads alone closes 53 percent of the gap to model B. Re-estimating batch norm alone closes 79 percent. And both together close 92 percent.

▶ **Step 31** — the repair-ladder table

In this table, A with zeros scores 0.664. A after the full repair, 0.868. Model B, 0.885. And at the bottom, a separate model trained for each lead setting, 0.905.

▶ **Step 32** — the summary text

So B still beats the repaired A, but only by about 0.02. And a dedicated model for each setting is another 0.02 above B.

State 4 · 7:55–9:25 · steps 33–40 · the key result

▶ **Step 33** — heading: *State 4 — the key result*

So what's the final answer?

▶ **Step 34** — panel: `seed_effects = [0.18, 0.2423, 0.1701]`

First, the test I fixed before I looked at the test labels: B minus A, averaged over the four reduced-lead settings. For the three seeds, that's 0.18, 0.24 and 0.17.

▶ **Step 35** — panel: `primary = 0.1974`

On average, 0.197, with a confidence interval from 0.188 to 0.207. So the main hypothesis holds.

▶ **Step 36** — the results table

And the gain grows as leads disappear: 0.12 with the six limb leads, and 0.30 with only leads one and two.

▶ **Step 37** — panel: `n_subsets`

But we've just seen that a cheap repair closes most of that gap. So I tested the repair on 207 different lead combinations, and split them by how many limb leads are left. 160 combinations have two or more, and 47 have fewer.

▶ **Step 38** — panel: `mean_gap`

With two or more limb leads, B and the repaired A are basically tied: a difference of 0.002. With fewer than two, the leads can't be calculated, and B is ahead by 0.071.

▶ **Step 39** — the subset figure

You can see it here. On the left, everything sits around zero. On the right, B is clearly ahead, most of all when there's only one lead.

▶ **Step 40** — the result text

And one more finding: masking more often works even better. Masking every training input reaches 0.898 instead of 0.885, with no loss on complete twelve-lead ECGs.

Close · 9:25–10:25 · steps 41–43

▶ **Step 41** — the healthcare table

In practice: if you can't predict which leads you'll get, train one model with masking. If a device always uses the same leads and you can retrain, a dedicated model is best. And if all you have is an existing complete-input model, calculate the missing leads and re-estimate batch norm, as long as two limb leads are there. In every case you still need to catch bad leads first: a lead that looks real but is wrong costs model B 0.235.

The lesson I'm taking away is this: an improvement only means something when you compare it with a strong, cheap baseline.

▶ **Step 42** — limits

This is one dataset, and the lead loss is simulated, so testing on real devices comes next. Thank you, and I'm happy to take questions.

▶ **Step 43** — presenter note (end of the talk; don't read it)

Likely questions

- **Why not always use the repair instead of masking?** The repair needs at least two limb leads and a set of batch-norm statistics for every lead combination, or the training data at deployment. Masking needs neither, and it clearly wins when fewer than two limb leads are left.
- **Did you cherry-pick record 514?** No. It's the first test record, by ID, whose only diagnosis is an inferior heart attack, chosen from the metadata before any model ran. I only use it to show the input; every model result is on the full test set.
- **Why only seed 17 in the live parts?** Running all three seeds would triple the build time. The three-seed numbers come from the saved results and are in the tables.
- **How did you use AI?** OpenAI Codex wrote and debugged the main training pipeline. Claude Code proposed and checked the extra analyses and drafted text and figures. A simulated peer review, run with an AI tool, proposed the stronger baselines, and in a later round it found a bug in my first subset analysis: the limb leads were only calculated when leads one and two were both present.
- **Is it ready for clinical use?** No. It's one dataset, the lead loss is simulated, and the labels are broad diagnostic classes. The next step is testing on real reduced-lead devices.